

StayBoost

Complete Project Audit Report

AI-Powered Hospitality Growth Operating System

Prepared by: CTO / Engineering

Date: 2026-06-16

Branch: claude/stayboost-product-blueprint-k3w0jp - Commits: 10

Scope: Full repository, read-only.

Headline

Phase 1 (Public Foundation) is 100% complete - 7 features across 3 services, 120 tests passing, clean builds.
Not yet provisioned against a live database or deployed.

1. Project Overview

What StayBoost currently does: a multi-tenant SaaS front door + intelligence layer for hospitality. A visitor can browse the marketing site, run a free AI Property Analyzer (no login), sign up (email/password, Google, or magic link), complete a Property Setup Wizard, and land in a Command Center dashboard with 8 widgets. It is the 'brain above the plumbing' - not yet connected to live PMS/OTA data.

Architecture (3 deployable services)

apps/web (Next.js 15) -> services/api (NestJS) -> services/ai (FastAPI). Web talks to the API via a typed client over httpOnly JWT + CSRF cookies; the API owns PostgreSQL + pgvector via Prisma; the AI service calls Claude with a heuristic fallback.

Tech Stack

Layer	Technology
Monorepo	pnpm workspaces + Turborepo, Node 22, TypeScript 5.6 (strict)
Web	Next.js 15 (App Router/RSC), React 19, Tailwind 3.4, shadcn-style UI
Core API	NestJS 10, Zod, RFC-7807 errors, Throttler, Helmet
AI service	Python 3.12 + FastAPI, Pydantic v2, Anthropic SDK (claude-opus-4-8)
Data	PostgreSQL + pgvector, Prisma 6 (RLS); Redis (rate-limit store)
Auth	bcrypt, JWT (HS256), opaque rotating refresh sessions
Email / CI	Resend (log-fallback); GitHub Actions (lint/typecheck/test/build + Python)

Folder Structure (10 workspaces, 243 tracked files)

- apps/web - Next.js app (marketing, auth, app route groups)
- services/api - NestJS core API | services/ai - FastAPI AI service
- packages/{domain, db, ui, api-client, i18n, utils, config}
- infra/docker-compose.yml (local Postgres+pgvector+Redis); docs/ (blueprint, ADRs)

2. Completed Features

#	Feature	Status	APIs	Models / UI
0	Monorepo foundation + CI	Done	-	config, CI workflow
1	Landing page	Done	-	/
2	Pricing page	Done	-	/pricing (plan catalog)
3	Contact (full-stack)	Done	POST /v1/contact	ContactLead; /contact
4	AI Property Analyzer	Done	analyzer/run, reports/:token, /v1/analyze	AnalyzerReport, AiAction; /analyzer
5	Authentication + RBAC	Done	14x /v1/auth/*	User, Org, Membership, Session, +3; 6 pages
6	Dashboard skeleton	Done	GET /v1/dashboard	reads Property; /dashboard
7	Property Setup Wizard	Done	POST/GET /v1/properties	Property; /onboarding

Every feature shipped through the same gate: tests -> typecheck -> lint -> build, then committed.

3. Authentication

Status: production-grade, fully implemented (backend + 6 web pages); 52 API tests cover it.

Login methods

- Email/Password (bcrypt cost 12)
- Google OAuth 2.0 (authorization-code + state CSRF; auto link/create + org)
- Magic Link (single-use, 15-min, hashed token)
- Email verification + Password reset (no account enumeration; reset revokes all sessions)

RBAC roles (hierarchical, org-scoped via Membership)

super_admin > owner > manager > staff - enforced by JwtAuthGuard + RolesGuard (@Roles, @CurrentUser, @Public).

Security measures

- Access = short-lived JWT (HS256, issuer/audience); refresh = opaque, hashed-at-rest, rotated with reuse-detection (theft -> revoke all sessions)
- All verify/reset/magic tokens hashed at rest, single-use
- httpOnly + Secure + SameSite cookies; double-submit CSRF on protected mutations
- Per-route rate limits, Helmet (CSP/HSTS), trust-proxy; full audit log
- Fail-fast production env (JWT_SECRET, AUTH_COOKIE_SECURE required)

4. AI Property Analyzer

Layer	Files
Frontend	app/(marketing)/analyzer/page.tsx; components/analyzer/{analyzer-form, growth-report-view}.tsx
Backend (NestJS)	analyzer/{controller,service,module}.ts; ai/{ai.client,ai.module}.ts
AI service (Python)	app/{main,analyzer,config,schemas}.py; engines/{claude,heuristic,base}.py

End-to-end flow

Web form -> POST /v1/analyzer/run (rate-limited, Zod) -> NestJS -> POST /v1/analyze (FastAPI, service-token) -> engine builds GrowthReport -> schema-validated -> NestJS stores AnalyzerReport (claim token) + logs AiAction -> report rendered + signup CTA.

Current capabilities

- 4-pillar scoring (Acquire/Monetize/Automate/Delight) with per-finding severity + action
- Revenue uplift as a transparent RANGE (never a guarantee), quick wins, confidence + provenance
- Claude (claude-opus-4-8, structured outputs, effort=high) primary; deterministic heuristic fallback (works with NO API key)

5. Dashboard

8 widgets implemented: Revenue Overview (sparkline + trend), Occupancy (radial + 7-day), Upcoming Check-ins, Upcoming Check-outs, AI Recommendations, Property Health Score, Recent Leads, Notification Center (unread count -> top-bar bell).

Shell: sidebar (full IA, future items marked 'Soon'), top bar (property switcher, avatar/logout), demo banner, loading skeletons, error/empty states.

Data source & remaining work

- Source: GET /v1/dashboard -> SampleDashboardProvider (deterministic, contract-validated). Now uses the real Property for name/type/units; widget metrics remain sample data.

- Remaining: replace sample provider with live Reservations/Revenue/Reviews; make sidebar sections real; wire 'Ask StayBoost'.

6. Database

Prisma 6, PostgreSQL + pgvector. Money = integer minor units; UTC; org_id on tenant tables + RLS.

11 models

Organization, User, Membership, Property, Session, VerificationToken, OAuthAccount, AuditLog, AnalyzerReport, AiAction, ContactLead.

Key relationships

- Organization 1-* Membership *-1 User; Organization 1-* Property
- User 1-* Session / VerificationToken / OAuthAccount
- AuditLog / AiAction / AnalyzerReport are platform-level (nullable org_id)

Migrations (4, hand-authored, CI-runable)

- 0001_init (contact_leads + pgvector); 0002_analyzer (analyzer_reports, ai_actions)
- 0003_auth (7 auth/tenant tables + RLS); 0004_properties (properties + RLS; relaxes memberships FORCE RLS)
- [!] Not yet applied to a live DB (Docker registry blocked) - RLS verified by design + mocks, not a running Postgres

7. API Endpoints

Method	Path	Auth
GET	/v1/health , /v1/health/ready	public
POST	/v1/contact	public (rate-limited)
POST	/v1/analyzer/run	public (rate-limited)
GET	/v1/analyzer/reports/:token	public
POST	/v1/auth/signup, login, refresh, logout	public (CSRF on refresh/logout)
GET	/v1/auth/me	JWT
POST	/v1/auth/resend-verification	JWT
POST	/v1/auth/verify-email, password/forgot, password/reset, magic-link, magic-link/consume	public (rate-limited)
GET	/v1/auth/google , /v1/auth/google/callback	public
POST	/v1/properties	JWT + RBAC(manager+) + CSRF
GET	/v1/properties	JWT
GET	/v1/dashboard	JWT
AI	GET /health , POST /v1/analyze	service-token

8. Testing

120 tests passing, 0 failing (verified this session).

Suite	Tests
services/api (auth, tokens, RBAC, CSRF, Google, analyzer, contact, properties, dashboard, env)	52

packages/domain	23
apps/web (forms, widgets, wizard, report view)	16
packages/utls 5 + api-client 4 + i18n 3 + ui 3	15
services/ai (pytest)	14
TOTAL	120

Coverage character: strong unit/contract coverage on domain logic, security primitives (hashing, rotation/reuse, RBAC, CSRF), schema conformance, and UI form paths. Gaps: no DB-backed integration tests yet, no executed cross-tenant RLS test, no e2e (Playwright), no AI eval harness.

9. Deployment Readiness

Production-ready now

Code quality (strict TS, no 'any'), security posture (auth/CSRF/secure cookies/audit/fail-fast env), validated boundaries (Zod/Pydantic), green CI gate, static-prerendered marketing/auth pages.

Missing for production

- Infra: managed Postgres+pgvector & Redis; migrations never run on a live DB; NO Dockerfiles, no vercel.json, no IaC for the two backend services
- Integrations not built: PMS/channel managers/OTAs, payments (Stripe/Razorpay), WhatsApp/SMS; Google OAuth & Resend need real credentials; captcha hook not wired
- Real data: dashboard widgets are sample data

Vercel readiness

apps/web is Vercel-deployable as-is (set NEXT_PUBLIC_*). services/api (NestJS) and services/ai (FastAPI) are NOT Vercel-native - host on Render/Railway/Fly/containers (Dockerfiles needed). Set COOKIE_DOMAIN/CORS for a shared parent domain across web<->API.

Required environment variables

- API: DATABASE_URL, REDIS_URL, JWT_SECRET, AUTH_COOKIE_SECURE=true, RESEND_API_KEY, APP_URL, WEB_ORIGIN, AI_SERVICE_URL, AI_SERVICE_TOKEN, GOOGLE_CLIENT_ID/SECRET/REDIRECT_URI, COOKIE_DOMAIN, EMAIL_FROM, CONTACT_INBOX
- AI: ANTHROPIC_API_KEY, ANTHROPIC_MODEL, AI_EFFORT, AI_SERVICE_TOKEN, AI_ALLOWED_ORIGINS
- Web: NEXT_PUBLIC_SITE_URL, NEXT_PUBLIC_API_URL

10. Phase Status

- Phase 1 (Public Foundation): 100% - 7/7 features delivered, tested, building
- Phase 2 (Operational MVP) readiness: ~20% - strong foundations (auth/RBAC, Property entity, AI seam, audit log, typed contracts), but no operational modules started

Remaining tasks to launch + enter Phase 2

- Provision DB/Redis + run migrations + cross-tenant RLS integration test
- Dockerfiles + deploy pipeline (api/ai to containers, web to Vercel)
- First channel/PMS connector (use onboarding Airbnb/Booking URLs) -> Reservations module -> replace dashboard sample data
- Analyzer prefill from saved property; smart inbox + auto-messaging; billing; e2e + AI eval harness

11. File Inventory (important files)

Area	Files & purpose
Domain contracts	packages/domain/src: auth, analyzer/{property-profile,growth-report}, dashboard, property-setup, billing/plans, contact - single source of truth, mirrored in Python
Data	packages/db: prisma/schema.prisma (11 models), migrations 0001-0004, src/index.ts (client)
Core API	services/api/src: main.ts, config/env.ts, common/{filter,pipe}, auth/* (15), properties/*, analyzer/*, ai/ai.client, dashboard/*, contact/*, health/*, prisma/*, mail/*
AI service	services/ai/app: main.py, analyzer.py, engines/{claude,heuristic,base}.py, schemas.py, config.py; tests/ (14)
Web	apps/web: app/(marketing auth app)/**, components/{marketing,pricing,contact,analyzer,auth,dashboard,onboarding,seo}, lib/{api,dashboard-format}, middleware.ts, SEO files
Design system	packages/ui: Button, Card, Badge, Input, Textarea, Label, Select, Skeleton + tokens
Docs	BLUEPRINT.md, CLAUDE.md, docs/00-10, docs/adr/0001-0002, phase-1-implementation-plan.md

12. Next Recommended Action

Build the data spine: provision the database, then ship the first Channel/Reservations connector that replaces dashboard sample data - in that order.

Why (CTO rationale)

- Provisioning + migrations + the cross-tenant RLS integration test is the single highest-risk unknown. Multi-tenant isolation (a P0 in CLAUDE.md) is not yet proven against a real Postgres - this must close before any customer data exists (~1-2 days, unblocks deploy).
- The wizard already captures Airbnb/Booking.com URLs - a connector that ingests reservations turns the dashboard and analyzer from sample/heuristic into live: the core value loop and the precondition for inbox, pricing, and reviews.
- Deferring CRM/Booking Engine is correct - both depend on reservation data existing first.

Concrete sequence

- (a) Managed Postgres+pgvector & Redis + prisma migrate deploy + RLS cross-tenant spec
- (b) Dockerfiles + deploy for api/ai; web -> Vercel
- (c) Reservation model + services/integrations connector (one channel) -> feed GET /v1/dashboard
- (d) In parallel quick win: analyzer prefill from the saved Property for logged-in users

Bottom line: a genuinely solid, secure, well-tested Phase 1. The right next move is to make it real - stand up the database (proving tenant isolation) and connect the first booking channel - before adding more surface area.